# Agent Evals Platform: Take-Home Assignment (Applied AI Engineer)

**Stack:** NestJS (control plane / API) · Vue 3 + TypeScript (UI). Harness execution can run in Python workers if that's more natural; the boundary between the two is up to you.

**Time box:** about 5 days. We care more about sound evaluation design than about feature count.

## Problem

We ship AI agents built from two parts: a **harness** (the loop, tools, skills, memory, prompting strategy) and a **model**. Either one can change at any time. We swap models, upgrade harnesses, add skills and turn on memory, and every change raises the same question: *did the agent get better, worse, or stay the same, and how sure are we?*

Build a platform that evaluates **any harness (X) running with any model (Y)** and answers that question with evidence.

## Harnesses and models

- **Required harness:** Hermes Agent, an open-source harness with skills, memory and tools.
- **Second harness:** add at least one more so we can see your abstraction holds. It can be a minimal baseline you write yourself (a plain tool-calling loop) or an existing framework (Claude Agent SDK, OpenAI Agents SDK, smolagents, LangGraph and so on).
- **Models:** at least two models from different providers, or one open-weight model, reached through a common interface (e.g. OpenRouter, LiteLLM, or your own adapter).
- Adding a new harness or a new model should mean writing an adapter, not changing the platform core. Show this in the design.

## What it should do

1. **Tasks and scoring.** Define a task set and how each task is scored. Support at least:
    - deterministic graders (exact match, unit tests, state checks on a sandbox or file system);
    - model-based graders (LLM-as-judge with a rubric).
2. Where a judge is used, show how you checked that it's trustworthy, e.g. agreement with a small set of human-labelled examples.
3. **Configurations.** Define what's being compared as **harness × model × settings** (skills, memory, tools, temperature, step and token budgets). Configs are versioned so a past result can always be reproduced.

4. **Execution.**
    - Run every task under every config, with **N repeated trials** per task, because agents are non-deterministic.
    - Runs are isolated: no state leaks between trials unless a memory setting explicitly says so.
    - Handle timeouts, rate limits, retries and partial failures without corrupting results.
5. **Comparison with confidence.**
    - Show whether config A beats config B, and how confident we can be: confidence intervals and a suitable paired test (e.g. bootstrap or McNemar).
    - Report per-task deltas, not just one aggregate score.
    - Make it clear when a difference is just noise.
    - Report **cost, latency and token usage** next to quality, so trade-offs are visible.
6. **Run inspection.** For any single trial, show the full trajectory: messages, tool calls and results, memory reads and writes, which skills were invoked, grader reasoning and the final score. A user should be able to see *why* it passed or failed.
7. **Failure analysis (stretch).** Group failures into categories (wrong tool, hallucinated tool args, gave up early, loop or step exhaustion, grader disagreement), so a regression points to a cause.

## Requirements

- TypeScript for the API and UI. Python is allowed for harness workers.
- Runs locally with one command (e.g. `docker compose up`), with API keys supplied through `.env`.
- Includes a **sample task set** of at least 20 tasks with a mix of grader types. At least a few tasks should be able to tell configs apart, e.g. tasks where a skill or memory should help.
- Results are persisted and can be exported (JSON or CSV).
- Document every assumption you make.

## Deliverables

1. **Source code** in a Git repository: production-grade, tested, with a README covering setup, how to add a task, how to add a harness, and how to add a model.
2. **Design document** covering:
    - the architecture;
    - the harness and model adapter interfaces;
    - the data model (tasks, configs, experiments, trials, traces);
    - how runs are isolated and reproduced;
    - grader design and how graders were validated;
    - the statistical method, and how you chose N.
3. **Known limits.** Cover where the platform's conclusions should *not* be trusted, e.g. task-set size, judge bias, contamination, and harness-specific quirks.

4. **Cost estimate.** Give the cost per experiment as a formula (tasks × trials × configs × average tokens × price), plus the actual spend from your experiment. Explain how you'd reduce cost (caching, early stopping, sampling).
5. **One real experiment** comparing two configurations that differ along a single meaningful axis: for example the same harness with two models, the same model in two harnesses, or Hermes with memory on vs. off. Include:
    - your hypothesis;
    - setup and number of trials;
    - results with confidence intervals;
    - two or three inspected trajectories that explain the difference;
    - your conclusion.
6. **Video** (≤10 min) of the working system: define a config, run it, compare results, drill into a failed trial.

## What we're evaluating

| Area | What we look for |
|---|---|
| Eval methodology | Sound grading, valid statistics, honest handling of noise |
| Abstraction | Clean harness and model adapters; a new X or Y is a small change |
| Observability | Traces detailed enough to debug an agent |
| Engineering quality | Reliability, tests, reproducibility, clear code |
| Judgment | Sensible scope cuts, clearly stated assumptions and limits |